BhoomiNITI AI

PLOTARMOR · SIH26019

PRODUCT WALKTHROUGH & REFERENCE DOCUMENT

What every screen shows, how it's built, and where it's going.

A page-by-page guide to the six operational views in BhoomiNITI AI, read through the lens of Bengaluru Urban — with the role-based access model explained in full, and an honest map of what runs on real logic today versus what's still mocked ahead of the FastAPI backend.

GIS CHANGE	EVIDENCE RAG	POLICY SCENARIO	BLOCKCHAIN PROVENANCE
Engine A	**Engine B**	**Engine C**	**Engine D**

CONTENTS

Reading this document

1. The problem, in one page	SIH26019
2. How the four engines fit together	A–D
3. Login & session setup	entry
4. Role-based access control (RBAC) — full explanation	RBAC
5. Land Intelligence — deep dive (Bengaluru Urban)	engine A
6. Evidence Search — deep dive	engine B
7. Policy Scenarios — deep dive (Bengaluru Urban)	engine C
8. Provenance Ledger — deep dive	engine D
9. Reports & Downloads — deep dive	output
10. Data & Sources	trust
11. What's done, what's mocked, what's next — full roadmap	roadmap

> **How to read the badges used throughout:**
> `REAL & WORKING` the logic genuinely runs (e.g. the PDF export). `REALISTIC MOCK` real district names / real cited documents, but the numbers or answer text are hand-authored, not computed from live data. `NEXT` planned once the FastAPI backend is connected.

SECTION 1

The problem, in one page

SIH26019, set by the Ministry of Rural Development's Department of Land Resources (DoLR) under the **Blockchain & Cybersecurity** theme, asks for a way to govern land use with *evidence* instead of assertion: detect how land is actually changing (farmland disappearing into built-up area, forests thinning, lakes shrinking), ground any policy discussion in citable documents rather than a bureaucrat's or an AI's unverified claim, let officials see the likely effect of an intervention before committing to it, and — because this is the Blockchain & Cybersecurity track — prove that none of the underlying data was quietly altered after the fact.

BhoomiNITI AI (Team PlotArmor) answers that with four purpose-built engines behind one workspace: a **GIS change engine** that quantifies land transitions between two years, an **Evidence RAG engine** that answers policy questions with citations instead of hallucinated confidence, a **Policy Scenario engine** that puts an observed trend next to a modelled intervention, and a **Blockchain Provenance engine** that hash-chains every dataset version so tampering is detectable. A single **Reports** layer packages all four into one exportable decision artifact.

> **Why Bengaluru Urban is used as the running example in this document:** its story is the most legible one the mock data tells — a near-static built-up share hiding a sharp, specific loss: notified water bodies (the "vanishing lakes of Bengaluru") converting into built-up and other land, concentrated around identifiable catchments like Bellandur–Varthur. It is a genuinely dramatic, well-known land-use story, not a generic placeholder.

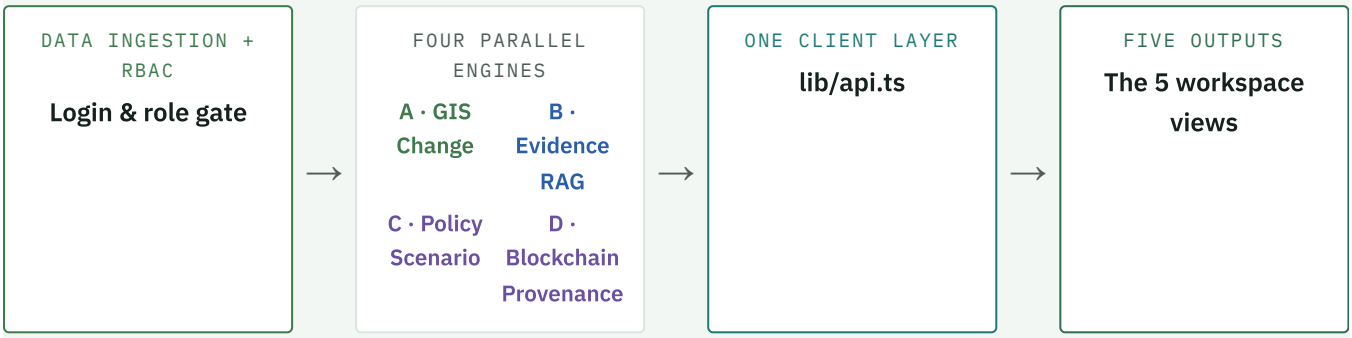
What this document is (and isn't)

This is a plain-language walkthrough of the six screens in the live app, written so you can explain any chart, table, or number on any page without having to reverse-engineer it from the code. It is not a pitch deck — where something is a hand-authored mock standing in for a future live computation, it says so explicitly, because that honesty is itself something worth being able to say confidently to a judge who asks "is this real?"

SECTION 2

How the four engines fit together

The product is one app shell, not four separate tools. Every screen you click through is a thin UI on top of one of the four engines, all reached through a single client-side integration point so the backend can be swapped in without touching a single component.



ENGINE	SCREEN IT POWERS	WHAT IT DECIDES / PROVES
A · GIS Change	Land Intelligence	Where, how much, and between which land-use classes did the district actually change
B · Evidence RAG	Evidence Search	What does the policy/research corpus actually say, with a page you can go check
C · Policy Scenario	Policy Scenarios	What would a specific intervention plausibly change versus doing nothing
D · Blockchain Provenance	Provenance Ledger	Has any dataset behind these three answers been altered since it was recorded
Synthesis	Reports & Downloads	Package A + B + C + D into one exportable, judge/officer-facing artifact

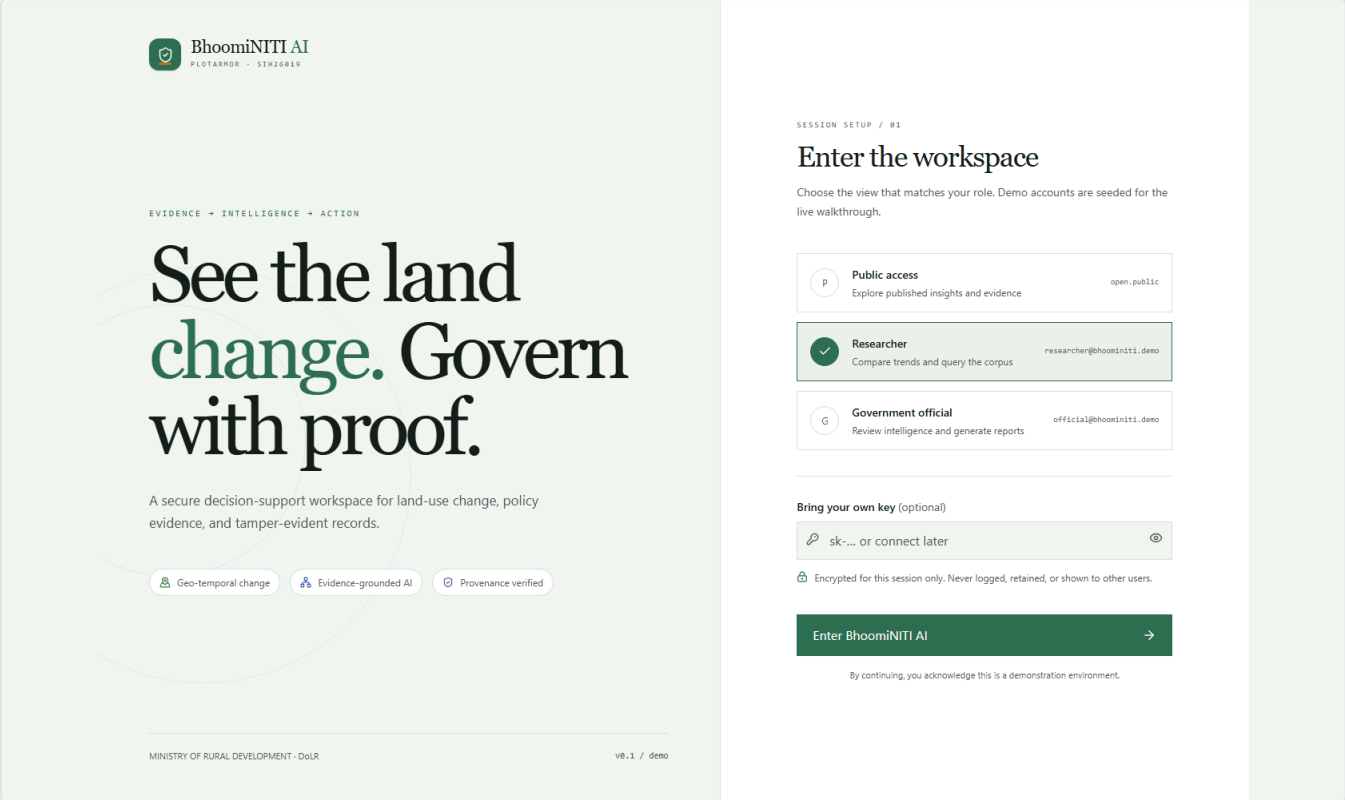
The frontend is Next.js 14 (App Router) with TypeScript and Tailwind CSS. Every one of the five engine-facing functions above is called through `lib/api.ts`: each function first tries a real HTTP call to `NEXT_PUBLIC_API_BASE_URL`, and — because that backend doesn't exist yet — falls back to a realistic, hand-authored mock response shaped exactly like the real one will be. That fallback pattern is what lets every screen in this document be fully demoable today while guaranteeing the eventual FastAPI swap touches one file, not six screens.

> **Honest framing used throughout:** "mock" here does not mean random or fake-looking. It means real Indian district names, real named government policy instruments, and internally consistent numbers that were deliberately authored to move correctly when you change the year range or district — but they are not yet the output of a live LULC classifier, a live retrieval pipeline, or a live forecasting model.

SECTION 3

Login & session setup

This is the judges' — and every user's — first click, and it does two jobs at once: it decides which role you're entering the workspace as, and it collects an optional AI key for the session.



The login screen: three seeded roles on the right, brand and value proposition on the left.

What's on screen

- Three role cards — **Public access**, **Researcher**, **Government official** — each with a seeded demo identity (`researcher@bhoominiti.demo`, `official@bhoominiti.demo`, or open public access).
- A "Bring your own key" field for an LLM API key, password-masked with a show/hide toggle.
- A one-line trust statement: the key is "encrypted for this session only. Never logged, retained, or shown to other users."
- "Enter BhoomiNITI AI" — routes straight into Land Intelligence.

How it's built

Selecting a role and clicking Enter calls `createSession()`, which in mock mode returns a session object — role, a demo token, whether a key was supplied, an expiry string — and that object is written to the browser's `sessionStorage`. The app shell reads it on every page load to decide what nav items to show.

`REALISTIC MOCK`

> **What the BYOK key actually does today:** the key you type is held in the login form's own memory only long enough to be handed to `createSession()`, which — in the absence of a backend — never transmits it anywhere; it simply records a true/false "a key was supplied." No live LLM call is wired to it yet. The key becomes functional the moment the FastAPI backend exists to receive, encrypt, and use it for the Evidence Search and Policy Scenario narratives — see Section 11.

Why a role selector belongs on the very first screen rather than behind a generic login: BhoomiNITI AI is deliberately **one shell with role-gated sections**, not three separate apps for three audiences. Deciding identity before anything else is what makes that single-shell model work — see Section 4 for the full access model this unlocks.

SECTION 4

Role-based access control — the full picture

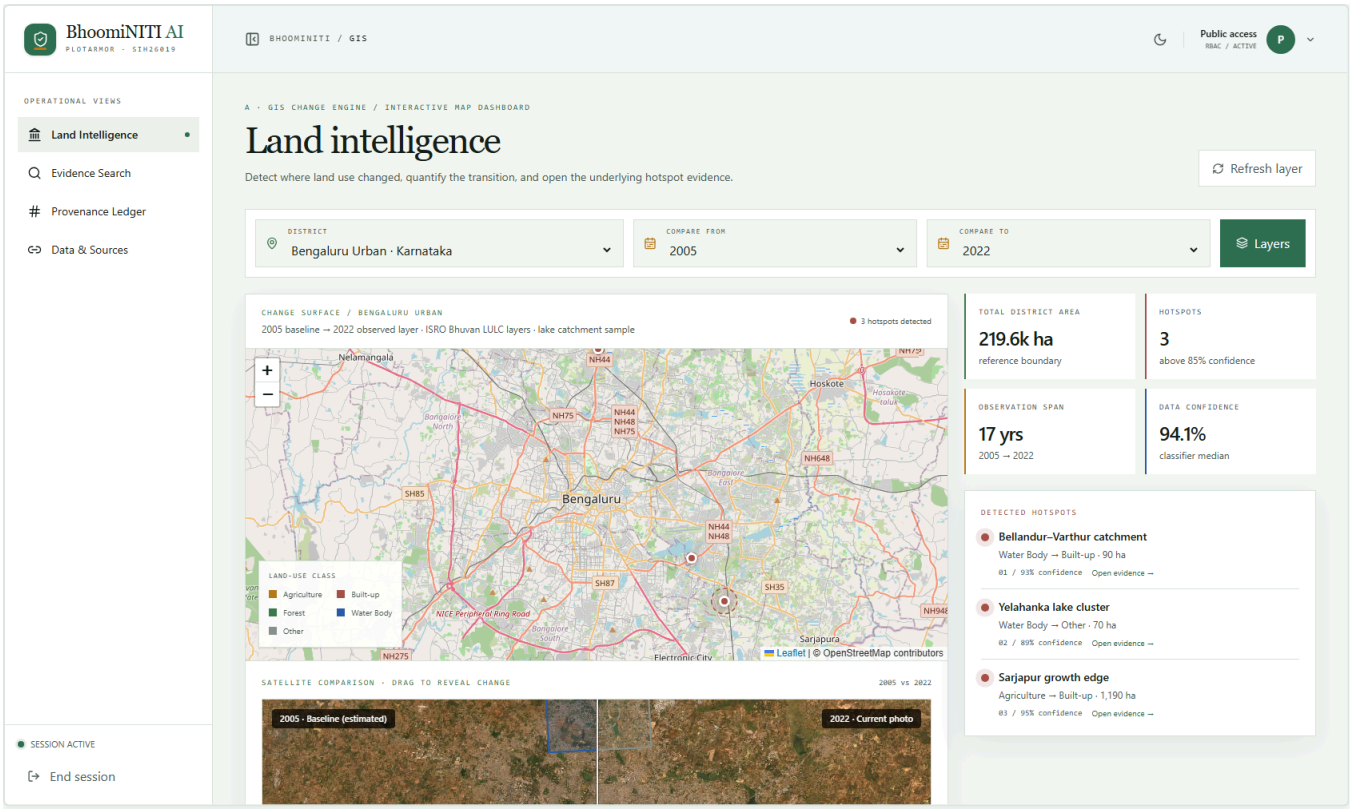
Every screen after login is gated by role. This section explains exactly what differs, why it differs, and — importantly — what "gated" honestly means in the current build.

What each role is for

ROLE	WHO IT REPRESENTS	INTENDED POSTURE
Public access	A citizen, journalist, or civil-society researcher with no credentials	Read-only transparency: see what changed, see the evidence, verify the provenance chain — cannot generate official artifacts or run "what-if" modelling
Researcher	An analyst formulating a recommendation — university, think-tank, or internal DoLR analytics staff	Everything Public gets, plus the modelling tools needed to build a case: scenario comparison and report generation
Government official	The person who actually acts on a recommendation — district authority, DoLR officer	Same surface as Researcher today; in the real system this is where approval/administrative actions would be added on top

What each role can see

SCREEN	PUBLIC	RESEARCHER	GOVERNMENT
Land Intelligence	✓	✓	✓
Evidence Search	✓	✓	✓
Policy Scenarios	–	✓	✓
Provenance Ledger	✓	✓	✓
Reports & Downloads	–	✓	✓
Data & Sources	✓	✓	✓



Same URL, same district, logged in as Public: the sidebar drops to four items — no Policy Scenarios, no Reports & Downloads. Compare against the Government view on the Land Intelligence page in Section 5.

Why the split falls exactly there — not a role per screen, a role per *kind of action*

Four of the six screens are pure transparency: they show what changed, what the evidence says, and prove the data wasn't tampered with. Making those Public-visible is the point of an evidence-based governance platform — a citizen should be able to verify a land-use claim without asking permission. The other two are different in kind, not just in sensitivity:

- **Policy Scenarios** produces a modelled, interpretive number — "280 ha protected under this intervention." Shown without the surrounding judgement of an analyst or official, a projection like that is easy to mistake for a decision that's already been made. Gating it to Researcher/Government keeps interpretation attached to someone accountable for it.
- **Reports & Downloads** produces an artifact formatted to look official — a "BhoomiNITI AI — Decision Support Report" PDF. Letting an unauthenticated public session generate that would blur the line between "a citizen ran a demo" and "the department issued a report." Restricting generation to authenticated roles keeps that boundary intact even though the underlying inputs (the map, the evidence, the ledger) stay fully public.

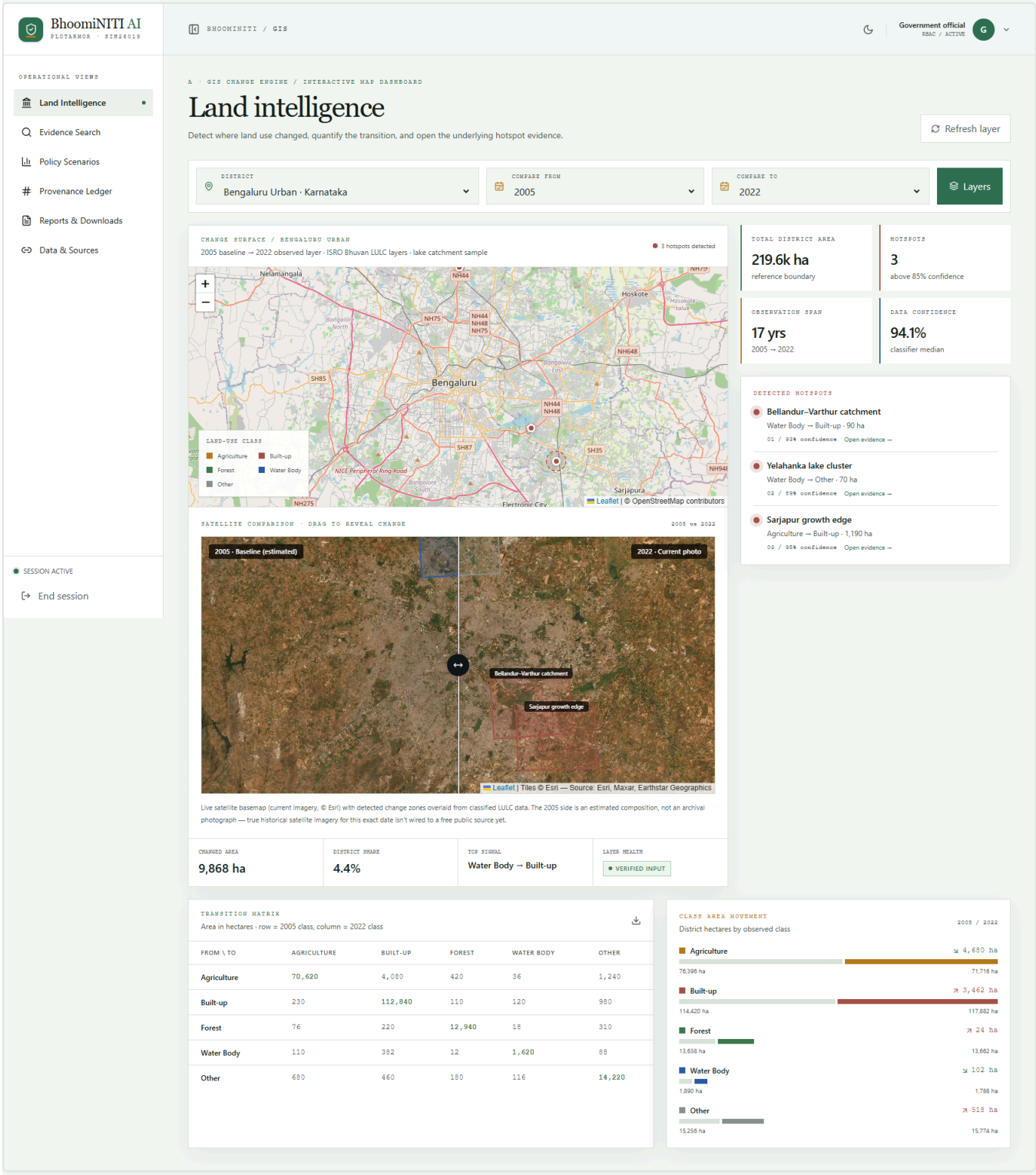
The benefit of doing this as one shell with a filtered nav array, rather than three separate apps, is that it's trivial to reason about and to demo: the exact same component renders for every role, so there's no risk of the Public build silently drifting out of sync with the Government build.

> **Honest limitation, worth being able to say out loud:** today this gate is *UI-level only*. The role list lives in the sidebar's navigation array (`AppShell.tsx`) and simply decides which links render — the `/policy-scenarios` and `/reports` pages themselves have no server-side permission check yet, so a technically savvy Public session could still type the URL directly. This is exactly the kind of gap the FastAPI backend needs to close with real, token-checked authorization on every endpoint — not just link-hiding. Naming this proactively is stronger than waiting for a judge to find it.

SECTION 5 · ENGINE A — GIS CHANGE ENGINE

Land Intelligence

This is the district-level "what actually changed" screen — a Leaflet map, a satellite before/after slider, a transition matrix, and a ranked list of hotspots, all driven by one district + year-range selection.



Bengaluru Urban, 2005 → 2022. Every number on this page recalculates when the district or year controls change.

Reading the top controls

Three dropdowns — **District** (Gurugram, Bengaluru Urban, or South 24 Parganas — the three districts chosen because each tells a distinct, well-documented land-use story), **Compare from** and **Compare to** (any pair of 2005 / 2010 / 2015 / 2022 / 2023 / 2024). Changing any of the three immediately refetches and redraws everything below — map hotspots, matrix, area bars, and the four summary stat tiles.

Reading the map and satellite slider

- **The Leaflet map** centres on the district (Bengaluru: 12.97°N, 77.59°E) with a land-use-class legend bottom-left — Agriculture (ochre), Built-up (brick red), Forest (green), Water Body (blue), Other (grey) — and a red pulsing marker at each detected hotspot.
- **The satellite comparison strip** below it is a draggable reveal: live current Esri satellite imagery on the right labelled "2022 · Current photo," an estimated 2005 composition on the left labelled "2005 · Baseline (estimated)," with the same hotspot boxes outlined on both sides so you can see the same patch of ground in each era. Dragging the black handle wipes between them.
- **Below the map**, four quick-read tiles: Changed area (**9,868 ha** for the full 2005–2022 span), District share (**4.4%** of Bengaluru Urban's 219.6k ha reclassified), Top signal (**Water Body → Built-up**, the single largest transition), and Layer health (a "Verified input" badge — a placeholder for the data-quality check a real ingestion pipeline would run).

Reading the right-hand rail

Four stat tiles — Total district area (219.6k ha), Hotspots (3, "above 85% confidence"), Observation span (17 yrs), Data confidence (94.1%, the classifier's median confidence across the district) — sit above the **Detected hotspots** list, ranked by severity: Bellandur–Varthur catchment (Water Body → Built-up, 90 ha, 93% confidence), Yelahanka lake cluster (Water Body → Other, 70 ha, 89% confidence), Sarjapur growth edge (Agriculture → Built-up, 1,190 ha, 95% confidence — the largest single hotspot, even though it isn't the water story). Each row is "Open evidence →" — the intended link from a detected change straight into the Evidence Search corpus for that transition type.

Reading the transition matrix

This is the table most people haven't seen before, so it's worth spelling out: **row = the 2005 class, column = the 2022 class**, and each cell is hectares. Reading the **Water Body** row for Bengaluru: 1,620 ha of what was water in 2005 was still water in 2022 (the diagonal — stayed the same); 382 ha became Built-up; 12 ha became Forest; 88 ha became Other. That single row is the entire "vanishing lakes" story reduced to four numbers. The diagonal cells (highlighted in green) are the "unchanged" mass; every off-diagonal cell is a specific, nameable conversion.

Reading the class-area-movement bars

Beside the matrix, one horizontal bar per land-use class: a grey segment sized to the 2005 hectares, a coloured segment sized to the 2022 hectares, with the net change and an up/down arrow. For Bengaluru: Agriculture ↓4,680 ha, Built-up ↑3,462 ha, Forest ↑24 ha (statistically flat), Water Body ↓102 ha, Other ↑518 ha. Red arrows mark classes moving in a direction that usually signals conversion pressure (Built-up, Other growing); green arrows mark classes shrinking in a way that's expected or benign.

How it's built

One call to `getLandChange({district, yearFrom, yearTo})`. In mock mode this runs `computeLandChange()` against a hand-authored per-class, per-marker-year hectare timeline for each district (not random noise — an internally consistent time series), scales the transition-matrix shape and hotspot magnitudes proportionally to whichever year span you picked, and returns the exact response

How this maps to the problem

This is the literal ask in SIH26019: detect land-use change over time — agriculture→built-up, forest loss, water-body shrinkage — using LULC classification, and make it spatially explicit rather than a paragraph in a report. It's also the evidentiary base every other screen leans on: Evidence Search cites the policy behind acting on a hotspot, Policy Scenarios models

shape the real endpoint will. The map is `react-leaflet`; the satellite layer is a live Esri tile layer with hand-placed hotspot overlay boxes.

`REALISTIC MOCK` `MAP RENDERING REAL`

what restraining one would do, Provenance Ledger proves the LULC tiles behind it weren't altered.

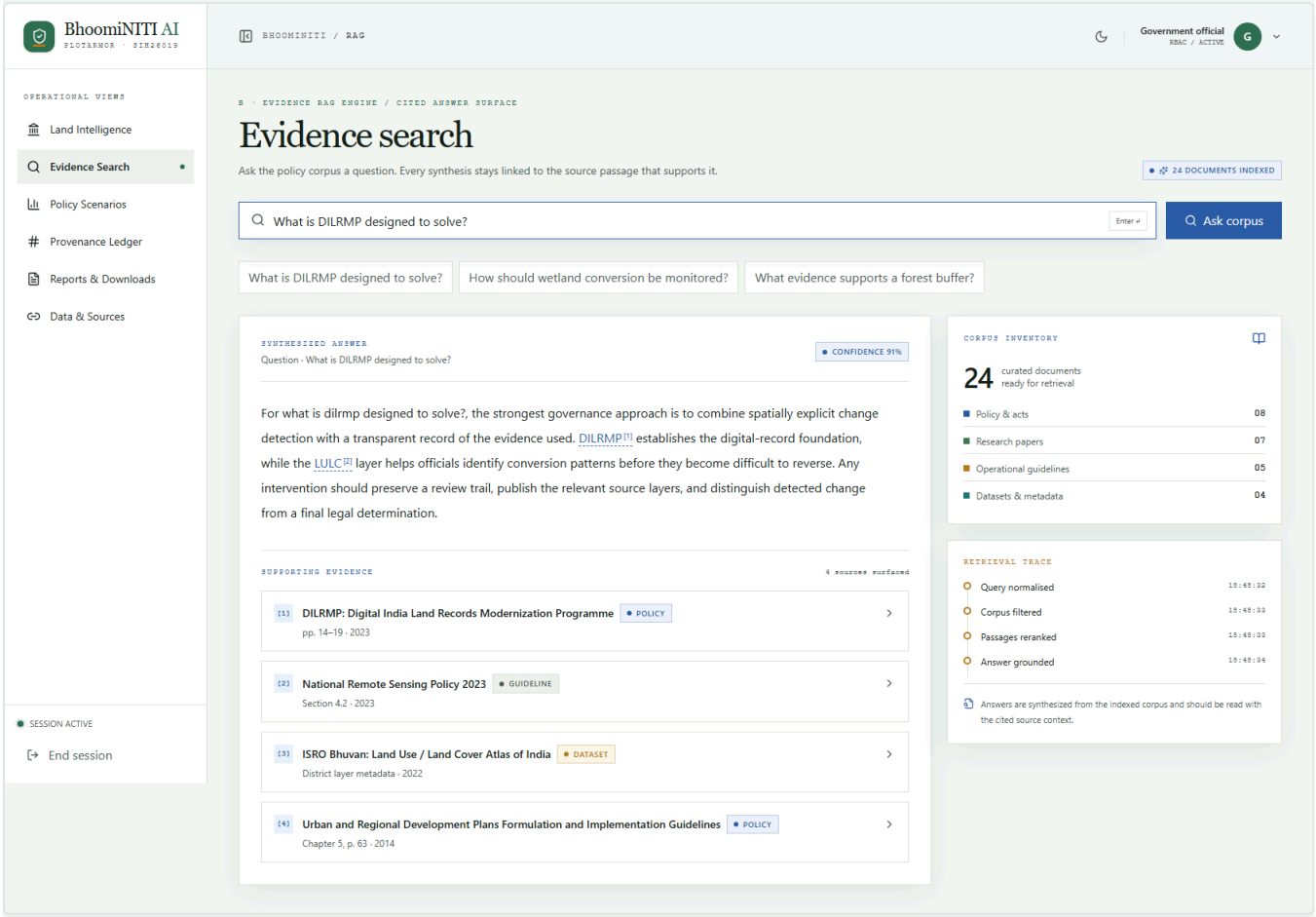
> **What's real vs mock, precisely:** the district, the coordinates, the land-use classes, and "ISRO Bhuvan LULC layers" as the named source are all real and correctly attributed. The hectare figures themselves are a plausible, hand-authored dataset, not a live classifier output — and the "2005" satellite image is explicitly labelled *estimated* because true archival imagery for that exact date isn't wired to a free public source yet.

Next: connect `GET /api/land-change` to real Bhuvan LULC rasters stored in PostGIS, run an actual multi-temporal change-detection classifier for the matrix and hotspot numbers, and source true historical satellite imagery for the "before" side of the slider.

SECTION 6 · ENGINE B — EVIDENCE RAG ENGINE

Evidence Search

A search bar over the policy/research corpus that returns a synthesized answer with inline citation markers — the mechanism meant to stop an AI answer from being trusted just because it sounds confident.



Query: "What is DILRMP designed to solve?" — answer with two inline citations, four expandable source cards, and the retrieval trace on the right.

Reading the answer panel

- The **synthesized answer** is a short paragraph with a **Confidence** badge (91% here) and inline citation markers — `DILRMP[1]` and `LULC[2]` — underlined in blue, each pointing at a specific numbered source below.
- **Supporting evidence** lists the sources that back the answer as clickable cards: title, a type badge (Policy / Guideline / Dataset), page/section reference, and date. Clicking a card expands a direct quoted snippet — e.g. source [1], DILRMP, pp. 14–19: "The programme prioritises conclusive titling, integration of textual and spatial records, and transparent mutation workflows."

Reading the right-hand rail

Corpus inventory shows the indexed document count (24) broken into four categories — Policy & acts (8), Research papers (7), Operational guidelines (5), Datasets & metadata (4). **Retrieval trace** is a four-step timeline — Query normalised → Corpus filtered → Passages reranked → Answer grounded — each with a timestamp, illustrating the actual stages a real retrieval-augmented-generation pipeline runs through between a question and a cited answer.

How it's built

One call to `searchEvidence(query)`. In mock mode it returns a fixed template paragraph with the query text substituted in, plus the first four of five hardcoded `EvidenceSource` records — so today the cited sources don't actually change based on what you ask; the retrieval step is illustrated, not executed.

`REALISTIC MOCK`

How this maps to the problem

This is the direct answer to "evidence-grounded AI search" in the brief, and it's arguably the single most important trust mechanism on the Cybersecurity/Blockchain theme: an AI answer that can't be traced to a specific page of a specific real document is exactly the kind of ungrounded claim this platform exists to prevent land governance from running on.

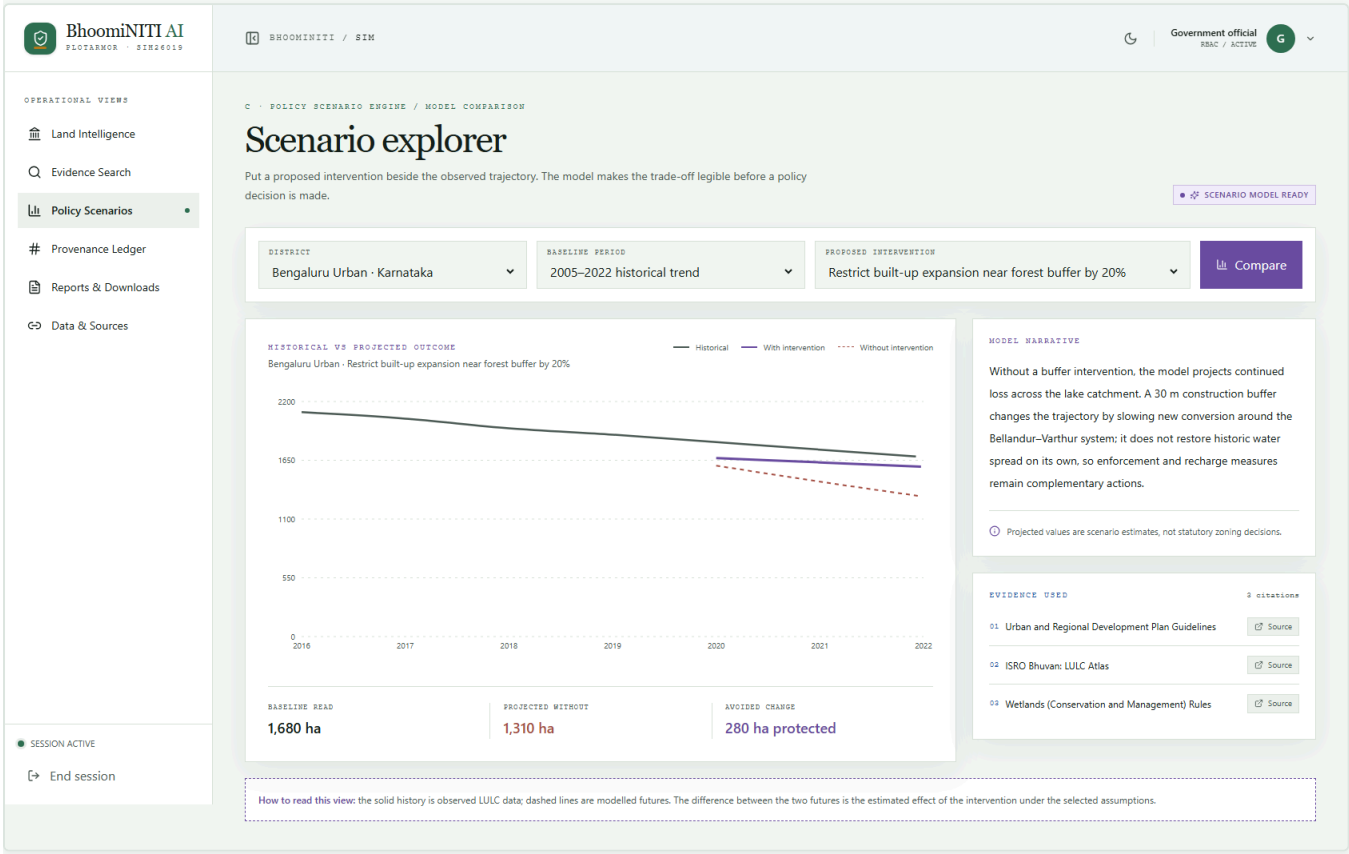
> **What's real vs mock, precisely:** all five underlying documents are real, correctly named government instruments — DILRMP, the National Remote Sensing Policy 2023, the ISRO Bhuvan LULC Atlas, the Urban and Regional Development Plan Guidelines, and the Wetlands (Conservation and Management) Rules. Their titles, types, and dates are accurate. What isn't real yet: the answer text isn't generated by an LLM, and the same four sources surface regardless of the question you type.

Next: ingest the full corpus, embed it with pgvector for real semantic retrieval, and route the synthesis through an actual LLM call using the BYOK key captured at login — so different questions genuinely surface different, re-ranked sources.

SECTION 7 · ENGINE C — POLICY SCENARIO ENGINE

Policy Scenarios

Puts the observed historical trend beside two modelled futures — with and without a proposed intervention — so the trade-off is a chart, not an argument.



Bengaluru Urban · "Restrict built-up expansion near forest buffer by 20%" — historical water-body decline vs. the modelled outcome with and without the intervention.

Reading the controls

Three dropdowns — **District**, **Baseline period** (2005–2022 full trend, or the steeper 2015–2022 acceleration window), **Proposed intervention** (four canned options, from a 30 m buffer around notified water bodies to requiring evidence review before land-use mutation) — then an explicit **Compare** button. Unlike Land Intelligence, changing a dropdown here does *not* auto-refresh the chart; you have to click Compare, which is a small but real UX detail worth knowing before a live demo.

Reading the chart

Three lines, and the legend above the chart tells you which is which: a solid grey line is **Historical** — actually-observed LULC data, 2016 through 2022. From 2020 onward it splits into a dashed red line, **Without intervention** (the projected continuation of the existing trend), and a solid purple line, **With intervention** (the projected trajectory if the selected policy were applied). For Bengaluru's water-body reading: historical falls from roughly 2,100 ha to 1,680 ha by 2022; projected without intervention keeps falling to 1,310 ha; projected with intervention only falls to 1,590 ha. The three metric tiles underneath restate this precisely — **Baseline read: 1,680 ha**, **Projected without: 1,310 ha**, **Avoided change: 280 ha protected** — that last figure being simply the gap between the two projected lines.

The dashed border callout at the foot of the page states the reading rule plainly: solid = observed, dashed/coloured = modelled, and the gap between the two modelled lines is the estimated effect of the intervention under the stated

assumptions — not a guarantee.

Reading the narrative and evidence panel

A plain-language paragraph explains the chart in words (for Bengaluru: "...a 30 m construction buffer changes the trajectory by slowing new conversion around the Bellandur–Varthur system; it does not restore historic water spread on its own, so enforcement and recharge measures remain complementary actions"), followed by an explicit disclaimer that this is a scenario estimate for review, not a statutory zoning decision. Below that, three evidence citations ground the scenario in real policy documents — though in the current build these three stay fixed regardless of which scenario you actually run.

How it's built

One call to `compareScenario({district, baseline, intervention})`. In mock mode, Bengaluru is specifically branched to a water-body-hectare series while every other district gets a built-up-percentage series — both hand-authored, not modelled from Engine A's actual detected-change velocity yet.

`REALISTIC MOCK`

How this maps to the problem

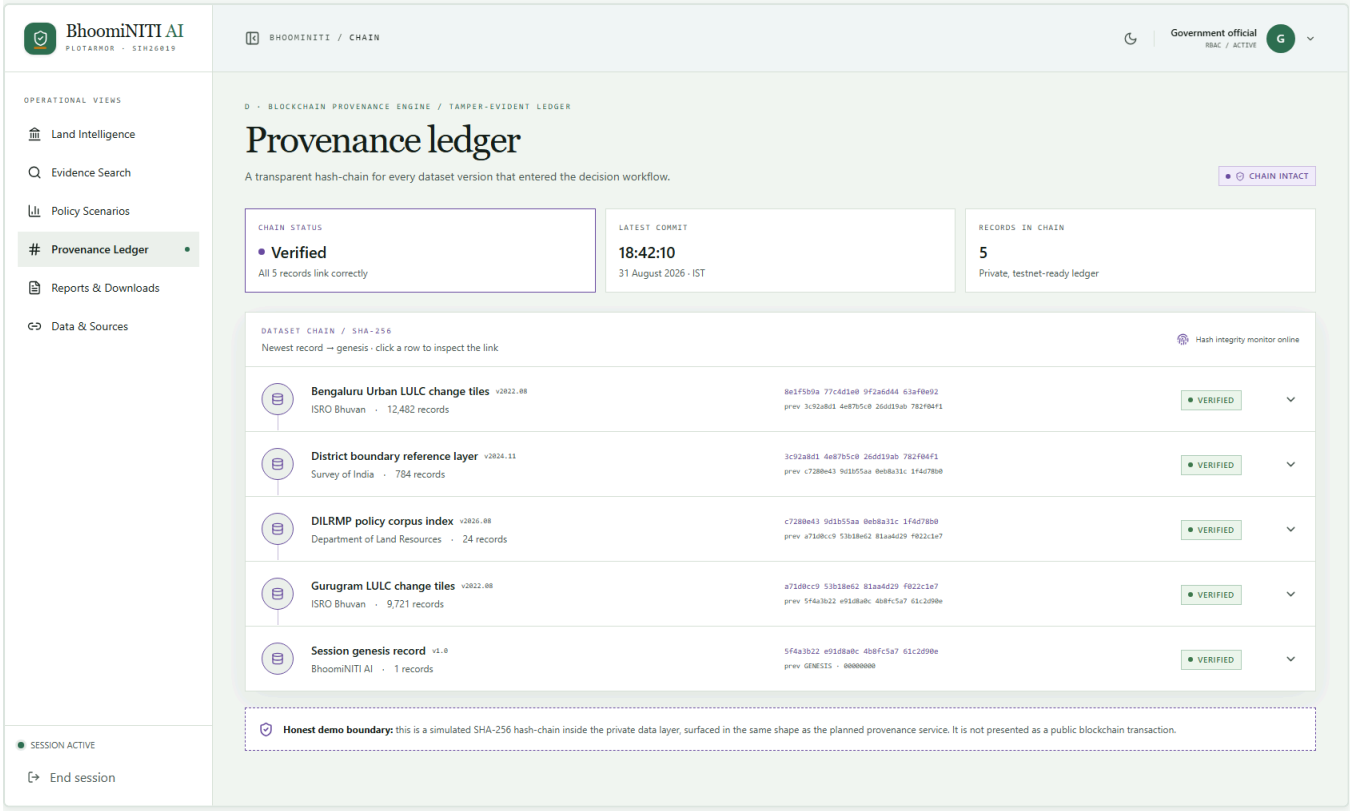
This is the "Policy Scenario Explorer" requirement verbatim: put a proposed intervention beside the observed trajectory so a trade-off is legible *before* a policy decision is made, which is precisely the kind of decision-support DoLR needs ahead of committing to a buffer, a restriction, or a review requirement.

Next: wire `POST /api/scenario/compare` to a real forecast grounded in Engine A's actual measured rate of change, and generate the narrative from the LLM (via the BYOK key) anchored to whatever Engine B actually retrieves for that intervention, rather than a static paragraph.

SECTION 8 · ENGINE D — BLOCKCHAIN PROVENANCE ENGINE

Provenance Ledger

The flagship screen for the Blockchain & Cybersecurity theme: a hash-chain proving every dataset version that fed a decision is exactly what it was recorded as, with no undetectable after-the-fact edits.



Five chained records, newest (Bengaluru Urban LULC change tiles) down to the session genesis record.

Reading the status tiles

Chain status: Verified — all five records link correctly. **Latest commit** — the timestamp of the newest entry. **Records in chain: 5** — described as "a private, testnet-ready ledger," i.e. not yet a public blockchain transaction, by design at this stage.

Reading the chain itself

Each row is one dataset version: name and version tag (e.g. "Bengaluru Urban LULC change tiles · v2022.08"), source and record count (ISRO Bhuvan · 12,482 records), a truncated SHA-256 hash, the previous record's hash it links back to, and a green "Verified" badge. Clicking a row expands the full hash, the full previous-hash, and a one-line proof statement: *"SHA-256(previous hash + payload) resolves to this record. Chain link verified at [timestamp]."* The mechanism this demonstrates is real and correctly modelled: each record's `previousHash` literally equals the hash of the record immediately above it, all the way back to a `GENESIS · 00000000` root — exactly how a real hash-chain proves tamper-evidence. If any past record's payload were altered, its hash would change, and it would no longer match what the next record down the chain references — breaking the "Verified" status immediately and visibly.

How it's built

`getProvenanceLedger()` returns five records with pre-computed, correctly-linked hash strings. The chain *structure* and verification logic are real; the hashes themselves are illustrative fixed strings rather

How this maps to the problem

This is the direct proof-of-integrity ask: government land decisions need to be defensible years later, which means the LULC tiles, the policy corpus index, and the boundary layers behind them need to be

than a live `SHA-256(previous hash + payload)` computed at ingestion time — the app itself is candid about this in the "Honest demo boundary" note at the foot of the page.

`MECHANISM REAL, HASHES ILLUSTRATIVE`

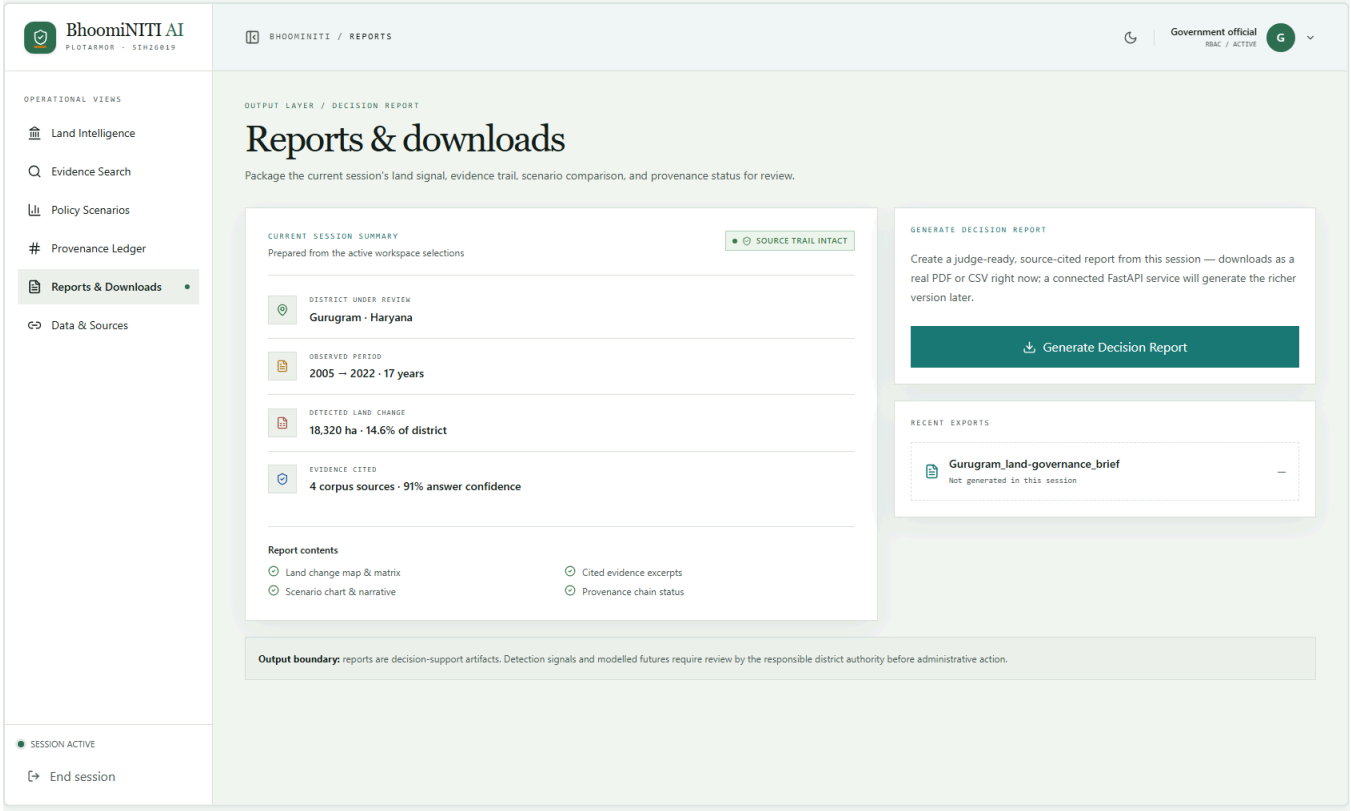
provably unaltered. This screen is where "evidence-based governance" becomes "auditable governance."

Next: compute real SHA-256 hashes server-side at the moment each dataset is ingested, persist them in an actual append-only store (or the ledger technology named in the system architecture), and expose a live verification endpoint rather than a static five-record list.

SECTION 9 · SYNTHESIS / OUTPUT LAYER

Reports & Downloads

Packages the session's findings — district, detected change, evidence, scenario — into one exportable decision artifact. This is the one screen where the export itself is genuinely, not just illustratively, functional today.



Session summary on the left, the Generate Decision Report action on the right.

Reading the session summary

Four rows restate the workspace state as a summary: District under review, Observed period, Detected land change (hectares and % of district), Evidence cited (source count and answer confidence) — with a "Source trail intact" badge tying it back to the Provenance Ledger. A checklist below confirms what the generated report will contain: land change map & matrix, cited evidence excerpts, scenario chart & narrative, provenance chain status — i.e. one artifact touching all four engines.

Reading the generate action

Clicking **Generate Decision Report** moves through a real loading state ("Assembling evidence trail...") to a success state with a report ID (e.g. `BN-2026-0831-014`) and two links: **Download PDF** and **Download CSV**. Both downloads are genuinely generated in the browser at that moment — the PDF is a real multi-page document (title, a data table, and an "Output boundary" disclaimer paragraph), the CSV is a real structured export — not a placeholder file.

How it's built

`generateReport()` returns a mock report ID; the PDF is then built entirely client-side with `jsPDF` + `jspdf-autotable`, and the CSV is hand-assembled and blobbed for download — both from a fixed `sessionSummary` object (currently hardcoded to a

How this maps to the problem

This is the tangible deliverable — the thing an official actually takes into a meeting, or a judge holds as proof the system produces a usable artifact, not just a dashboard. Every export carries the same "decision-support, not a final legal determination" boundary

Gurugram example, not yet synced to whatever district you actually explored in Land Intelligence during the session).

`PDF / CSV GENERATION REAL`

`SUMMARY DATA FIXED`

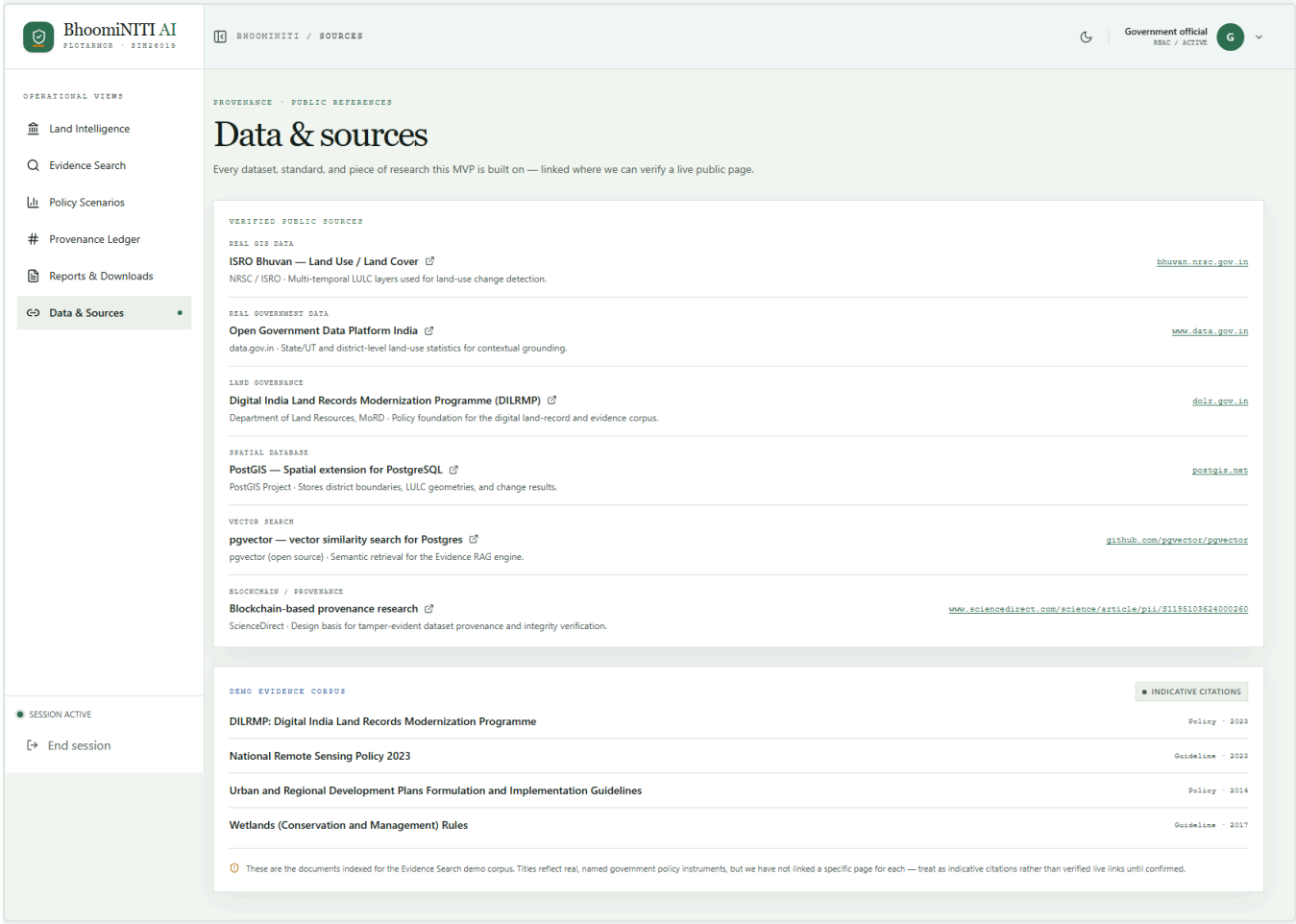
language the app repeats throughout, which matters for a platform meant to inform rather than replace administrative judgement.

Next: replace the hardcoded `sessionSummary` with the live selections actually made across the session, then move report assembly server-side via `POST /api/reports/generate` for a richer layout that can embed the real provenance hash and, eventually, a digital signature.

SECTION 10

Data & Sources

A dedicated transparency page listing exactly what this build is grounded in — and, just as importantly, drawing a clear line between what's a verified live link and what's an indicative citation.



Six verified public sources with live outbound links, plus the four-document demo corpus marked "Indicative citations."

Reading the two lists

- **Verified public sources** — six real, clickable references: ISRO Bhuvan (LULC data), the Open Government Data Platform (data.gov.in), DILRMP (dolr.gov.in), PostGIS, pgvector, and a ScienceDirect paper on blockchain-based provenance — each tagged with the category it supports (GIS data, government data, land governance, spatial database, vector search, blockchain/provenance) and how it's actually used in the architecture.
- **Demo evidence corpus** — the same four documents cited throughout Evidence Search and Policy Scenarios, explicitly badged **"Indicative citations"** with a note that their titles are real, named instruments but no specific verifiable page has been linked for each yet.

How this maps to the problem

This page is itself a piece of the trust story rather than just documentation: it's the platform being checkable about its own claims, in the same spirit as the Provenance Ledger being checkable about its data. For a judge deciding whether to believe the rest of the app, this is the page that lets them verify instead of trust.

Next: link every demo-corpus document to its exact verifiable government gazette/PDF URL, and grow the verified list as the real Bhuvan/data.gov.in integrations land.

SECTION 11

What's done, what's mocked, what's next

Done — genuinely working today

- A complete Next.js 14 (App Router) + TypeScript + Tailwind frontend, responsive on desktop and mobile/tablet, with full light and dark theme support via CSS tokens.
- One typed API client (`lib/api.ts`) whose six functions already match the intended FastAPI contract exactly — `auth/session`, `land-change`, `evidence/search`, `scenario/compare`, `provenance/ledger`, `reports/generate` — so connecting the real backend later is a one-file change, not a rewrite.
- Internally consistent, hand-authored (not random) mock datasets for three real Indian districts, each chosen for a genuinely distinct, well-documented land-use story.
- Role-gated navigation across six operational views for Public / Researcher / Government.
- Real, functioning client-side PDF and CSV report generation — not a placeholder download.
- A deliberately non-templated, judge-oriented visual system (Fraunces + IBM Plex pairing, a fixed house palette, restrained motion) built specifically to avoid the generic-AI-dashboard look.

Next — what connecting the backend unlocks

AREA	CURRENT STATE	NEXT STEP
Land Intelligence	Hand-authored hectare timeline	Real Bhuvan LULC layers in PostGIS + a live change-detection classifier
Evidence Search	Fixed answer template, 5 static sources	Real corpus ingestion, pgvector embeddings, live LLM synthesis via the BYOK key
Policy Scenarios	Hand-authored projected series	A real forecast grounded in Engine A's measured rate of change, LLM narrative anchored to live retrieval
Provenance Ledger	Correct chain mechanism, illustrative hash values	Live SHA-256 computed at ingestion, persisted in an append-only store
Reports	Real PDF/CSV, but from a fixed example summary	Sync to live session selections; move assembly server-side with embedded provenance hash
RBAC	UI-level nav filtering only	Server-side, token-checked authorization on every endpoint

> **The throughline:** every mock in this build was authored to be structurally identical to what the real answer will look like — same field names, same response shape, same realistic magnitudes. That's what makes the "what's next" column above a swap, not a rebuild.